Solution Design: Cold-Chain Alerting Engine (Nimbus Freight)

1. The problem in my words

Every reefer truck sends a temperature reading every ~5 minutes. Today ops gets an alert for every warm reading, so they get swamped, ignore the alerts, and miss real failures. Nimbus wants **one alert per real, sustained problem on a specific truck**. That means ignoring brief blips, not re-alerting while the same problem continues, and getting it right even when readings go missing or arrive out of order.

2. What the seed data shows

54 readings, 5 trucks, 08:00–08:50 on 2026-07-20. Each truck tests one thing:

Truck	What happens	Starter code	My engine
T-01	Above 8 °C from 08:10 to 08:40, peak 9.6 °C	Alert	**Alert** at 08:30, closed at 08:50
T-02	One reading of 8.9 °C at 08:10	No alert	No alert (blip)
T-03	Climbs to 7.0 °C, never out of range	No alert	No alert
T-04	5-min blip at 08:05, then above 8 °C from 08:25 to 08:45; **08:35 reading missing**	**Missed** (counts 4 readings × 5 = 15 min)	**Alert** at 08:45 (20 min by timestamps); still recovering at 08:50
T-05	Same values as T-01, **shuffled order in the file**	**Missed** (splits it into 0/5/15-min pieces)	**Alert** at 08:30 after sorting

Result: 3 alerts, 3 notifications in total, and a reason for every truck, including why T-02 and T-03 did *not* alert.

3. Assumptions

Full list with reasons: `docs/ASSUMPTIONS.md`. All numbers live in one config file (`backend/engine/config.py`).

- **Safe range 2–8 °C** for the whole fleet (the standard range for refrigerated medicines, Nimbus's strictest cargo; every reading fits it). Above 8.0 or below 2.0 is out of range; exactly 8.0 is fine. There's no cargo-type field, so **every truck defaults to the strictest range**. Ranges are not hard-coded: **every truck has its own row in the database**. The first time a truck reports, it is registered automatically with an **empty range**, and an empty range means the default applies, so a new truck is never unprotected. Ops can fill in a truck's own range (and alert time) later from the dashboard, or send it with the truck's first upload. Changes need the admin key, and the fleet is re-evaluated immediately.
- **Time comes from the sensor timestamp**, not from when we received the reading. Timestamps without a zone are treated as UTC.
- **Alert after 20 minutes out of range** (the same for every truck in v1). On the seed data any limit from 10 to 20 min gives the right answer: 5 min alerts on T-04's early blip, and 25 min misses T-04. I picked 20, the most patient value that is still correct. It ignores door openings and defrost cycles (5–15 min), still has 3 real readings if one is missing, and leaves ops time to act before pharma cargo spoils.
- **Escalate to Critical after 60 minutes out of range, or 3 °C or more out of range (≥ 11 °C).** A truck still out of range after an hour means the first alert didn't get the problem fixed, so it goes further up. A truck that hot is probably a failed reefer unit, so it doesn't wait for the hour: it still needs the 20 minutes (one hot reading could be a glitch), then opens straight as Critical.
- **Between opening and Critical, the data decides whether to send an update; there are no timed repeats.** Every 10 min while the alert is open and the truck is still out of range, the current temperature is compared with the one 10 min earlier. If it hasn't improved by at least 0.3 °C (more than sensor jitter), one update goes out: "T-04 still above 8 °C after 30 min and not improving (9.1 → 9.4 °C)". If it is improving, back in range, or already Critical, nothing is sent. That's at most one update per 10 min, only for a truck nobody has fixed, and it closes the 40-minute silent stretch between the alert (20 min) and Critical (60 min). On the seed data every alert was already improving, so there are no updates (still 3 notifications).

- **A truck is back to normal only after 10 minutes in range**, confirmed by at least 2 readings. One dip just under 8 °C doesn't reset anything. This applies before the alert (otherwise a truck reading 8.5 / 7.9 / 8.5 / 7.9 would never alert) and after it (otherwise the alert flaps open/closed).
- **Missing readings:** we never make up a temperature. A gap of up to 15 min keeps the episode going (T-04). A longer gap is bridged only if the truck is out of range on both sides, and the alert says "includes missing data". Otherwise a not-yet-alerted episode restarts. **An open alert never closes because of missing data**, only on confirmed in-range readings.
- **A silent sensor is not treated as a hot truck.** That would bring back the noise, send ops to fix the wrong thing, and put made-up data in a safety record. It gets its own status and notification. After 15 min with no data the page shows "sensor silent". After **30 min** one notification goes out. If the truck was already out of range when it went quiet, the notification goes out at **15 min** as Critical, because a warm truck we can't see is the riskiest case. In the seed file "now" is the latest timestamp (08:50) and every truck reports up to then, so none is silent. Tests cover it.
- **Duplicates** (same truck and timestamp) are dropped, keeping the first. **Impossible values** (< −40 °C or > 50 °C) are flagged as sensor faults and not used. Malformed readings are rejected with a reason.
- **Humidity is stored and shown but doesn't trigger alerts.** In this data it moves with temperature: the correlation is 0.89 across the fleet and 0.88–0.98 per truck, with a range of 48–62 %. A humidity alert would duplicate the temperature alert and add noise, and the brief gives no humidity limit. That's from 54 synthetic readings over one hour, not proof for real trucks: wet cargo, condensation or a leak can raise humidity on its own. So it stays available to explain alerts ("humidity also rose 55→62 %, suggesting warm air getting in"), and moisture-sensitive cargo could get an optional humidity limit in its own range settings.

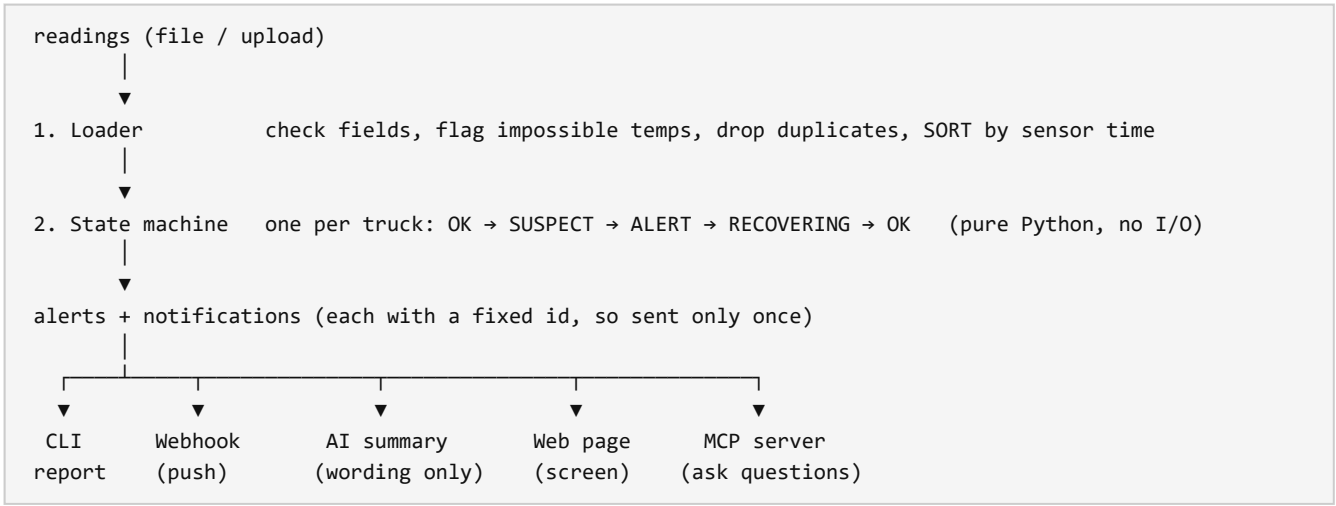
4. Why sorting is needed

The rules depend on order: "out of range for 20 minutes *in a row*" only makes sense if readings are in time order. Trucks lose signal and upload in bursts, and retries arrive late, so the order they arrive in is not the order they were taken in. T-05 shows the failure. Read in file order (08:00, 08:05, 08:20, 08:45, 08:35, 08:10, …), it seems to go warm, cool, warm, so one 30-minute problem looks like three short ones and **no alert fires**. Sorted by sensor time, T-05 is the same as T-01 and alerts. A test shuffles the whole file 5 ways and gets the same result each time.

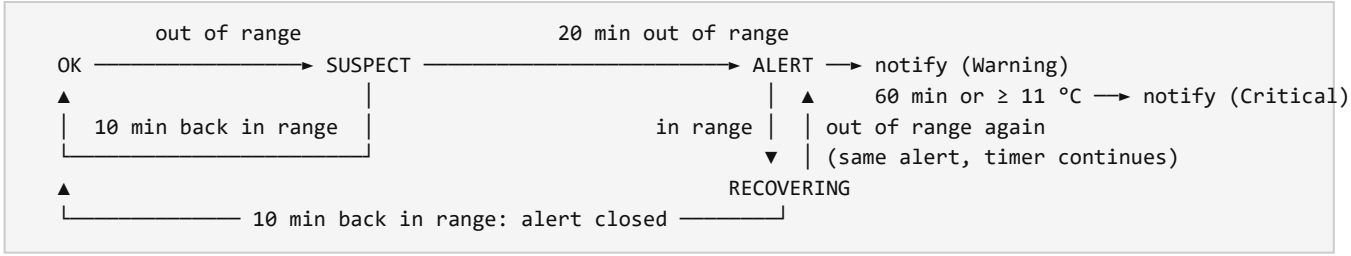
- **v1 (whole file):** group by truck, sort by timestamp, drop duplicates, then run the rules.
- **Live:** a late reading is put in its correct place and that truck is recalculated, so the result doesn't depend on arrival order. Readings more than 60 min late are logged but don't change alerts. An alert already sent is never deleted; it is marked "revised" with the reason.

5. The design

The whole system. The rules decide, and everything else only presents or explains what they decided:



The state machine (per truck):



State	Meaning	Notifies?
OK	In range	No
SUSPECT	Out of range, timer running	No (T-02 and T-04's early blip end here)
ALERT	20 min reached; duration and peak keep updating	**Once** when it opens; an update every 10 min **only if not improving**; **once** if Critical
RECOVERING	Back in range, waiting 10 min before closing	No

"Sensor silent" is a separate flag shown next to the state, so a dead sensor never hides or fakes a temperature alert. It has its own single notification and clears when data comes back. Short episodes that never reached 20 minutes are kept too, so the report can say *why* a truck didn't alert ("one reading out of range (8.9 °C at 08:10), under the 20-min limit").

6. How ops hears about it

There are three channels, each with one job:

Channel	Direction	Job
Web page	Ops looks at it	The main screen: every truck's state, open and closed alerts with reasons, temperature chart per truck. Refreshes automatically.
Webhook (Discord / Slack channel, e.g. `#ops-alerts`)	We push to ops	Reaches ops when nobody is looking at the page. Sent only when an alert opens, isn't improving after another 10 min, turns Critical, or a sensor goes silent.
MCP server	Ops asks an AI	"Which trucks are in trouble right now?" Read-only (§7). It doesn't send anything.

Webhook details. The notifier POSTs JSON to `ALERT_WEBHOOK_URL` with both `text` (Slack) and `content` (Discord), plus the alert's fields. Example: "*WARNING: Truck T-04 above 8 °C for 20 min (08:25–08:45), peak 9.1 °C. Includes missing data: 08:30–08:40 (10 min).*"

- **No repeats:** each notification has a fixed id (e.g. `T-04:2026-07-20T08:25:00+00:00:opened`), and sent ids are saved, so nothing is sent twice, even after a restart.
- **Failures:** 5-second timeout and 3 retries with growing waits. If it still fails, we log it and carry on. The alert stays visible on the web page.
- **Security:** the URL is a secret, because its token is part of the path (`.env` and Secret Manager only, never committed or logged). HTTPS only. Errors never echo it, and the HTTP library's own request logging is turned down, because a test caught it writing the full URL to the log. The message holds truck id, times and temperatures, with no customer or cargo details.
- If no URL is set (and in all tests), notifications go to the console and log.
- **Why not SMS:** SMS costs money, needs a registered sender in India, and would add a second noisy channel. The notifier has one interface with swappable senders, so SMS for Critical alerts only can be added later without touching the engine.

7. AI features (bonus): rules decide, AI explains

- **Alert summary:** an LLM (Groq) turns an alert's numbers into one plain sentence with a *likely* cause, e.g. "steady rise with humidity up, so likely a reefer problem rather than a door opening". Only numbers go in (truck id, times, temperatures, humidity), never customer or cargo data. The answer is checked: it must name the right truck, stay short, and contain no temperature or time that wasn't in the alert. If the check fails, or the call takes more than 5 s or errors, the plain rule-based message is sent instead, so an alert is never delayed or lost because of the AI. One call per alert, cached. Shown labelled as "AI summary", next to the exact numbers. Tests run in mock mode.
- **MCP server:** read-only tools (`get_fleet_status`, `get_open_alerts`, `get_truck_history`, `get_alert`) over the same engine, so an ops manager can ask an AI assistant "which trucks are in trouble right now?" and get answers built from real data. No tool can close or silence an alert; that stays a human action. Inputs are checked (known truck ids, capped time windows). It runs over stdio. It is used three ways: from Claude Code (`.mcp.json`), from the MCP Inspector, and from the dashboard's **Ask about the fleet** box, where our backend is the MCP client: it starts the MCP server (pointed at the live API), gives Groq the tool list, runs the tools Groq picks through MCP, and checks that every number and time in the answer appears in the tool results before showing it. The Ask box is read-only, needs no key, and is limited to 20 questions a minute.

8. Technology

Part	Choice	Why
Engine and CLI	Python 3.12 locally (Cloud Run builds with 3.13), plain functions plus a config file	Simple, easy to test, runs without a server
API	FastAPI + Pydantic	Validates uploaded readings; `/health`, `/api/trucks`, `/api/alerts`, upload endpoint
Storage	SQLAlchemy + PostgreSQL 16 (Cloud SQL in production)	Keeps alert history and sent-notification ids across restarts; portable SQL with bound parameters
Webhook	httpx + tenacity	Timeouts and retries out of the box
Web page	React + TypeScript (Vite)	The build fails if the page and the API disagree about the data shape, rather than showing a blank screen in the demo
AI	Groq LLM (Gemini as fallback) + official `mcp` Python SDK	See §7; mock mode in tests
Tests	pytest (engine, CLI, API, webhook with a fake sender), vitest (UI)	One test per truck story plus every edge case
Deploy	Google Cloud Run (asia-south1) + Cloud SQL; Azure App Service as backup	Already set up; secrets come from Secret Manager

9. Trade-offs I considered

- **Alert on every reading** (today): simple, but it causes the noise problem.
- **Rolling average:** smooths out blips but hides short, sharp spikes, and still flaps around the limit.
- **ML anomaly detection:** 54 readings isn't enough to train on, and ops can't see *why* it alerted.
- **Chosen, a duration-based state machine:** easy to explain ("20 minutes above 8 °C"), predictable, testable, and each number can be tuned per cargo.
- **Costs of this choice:** an alert can't fire sooner than 20 minutes, and a truck hovering near 8 °C will alert (on purpose, for pharma). A 20-minute episode sits exactly on the limit (T-04), so we test that edge explicitly; with a 25-minute limit T-04 would be missed.
- **AI kept out of the decision:** the LLM could spot patterns, but an alert must be explainable and repeatable. So the LLM only words messages, and every number it writes is checked.

10. With more time, data or budget

- **Ranges per shipment:** today ranges are set per truck by ops. Ideally the range would come with each shipment from Nimbus's dispatch system, plus an optional humidity limit.

- **Sensor-fault notification:** today impossible readings are flagged and ignored, but nobody is notified, and a truck sending *only* faulty readings drops out of the report. Fix: a "sensor fault" status with one notification after 15 min of bad readings.
- Streaming ingestion (Pub/Sub) instead of file upload; late-data handling as in §4; on-call escalation (who gets an update if nobody fixes the truck).
- "Trending warm" early warning (T-03's slow climb), and door-open or defrost signals from the truck to explain blips.
- SMS for Critical alerts only; a hosted MCP server with authentication.

Author: T V Sai Pranav (22CS02006) · **Live:** https://cold-chain-alerts-78623167451.asia-south1.run.app · **Code:** https://github.com/PRANAVTANGUTURU123/22CS02006_HCLTECH

11. Screenshots

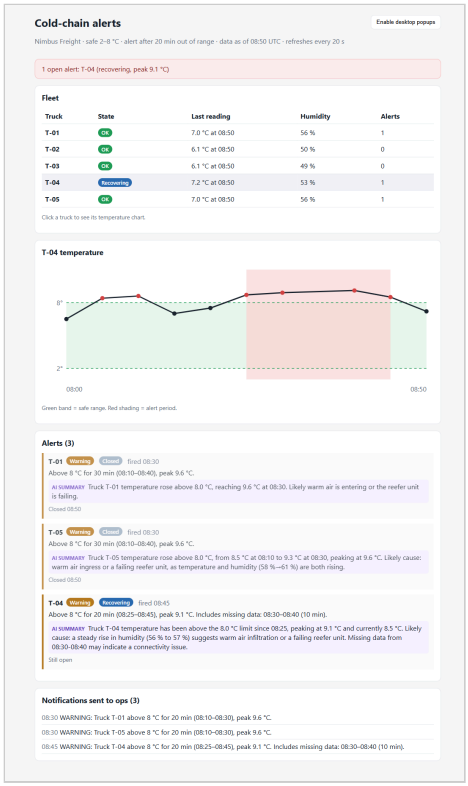


Figure 1. Ops dashboard: fleet states, T-04's chart with the 2–8 °C band and the alert period shaded, alerts with reasons and labelled AI summaries.

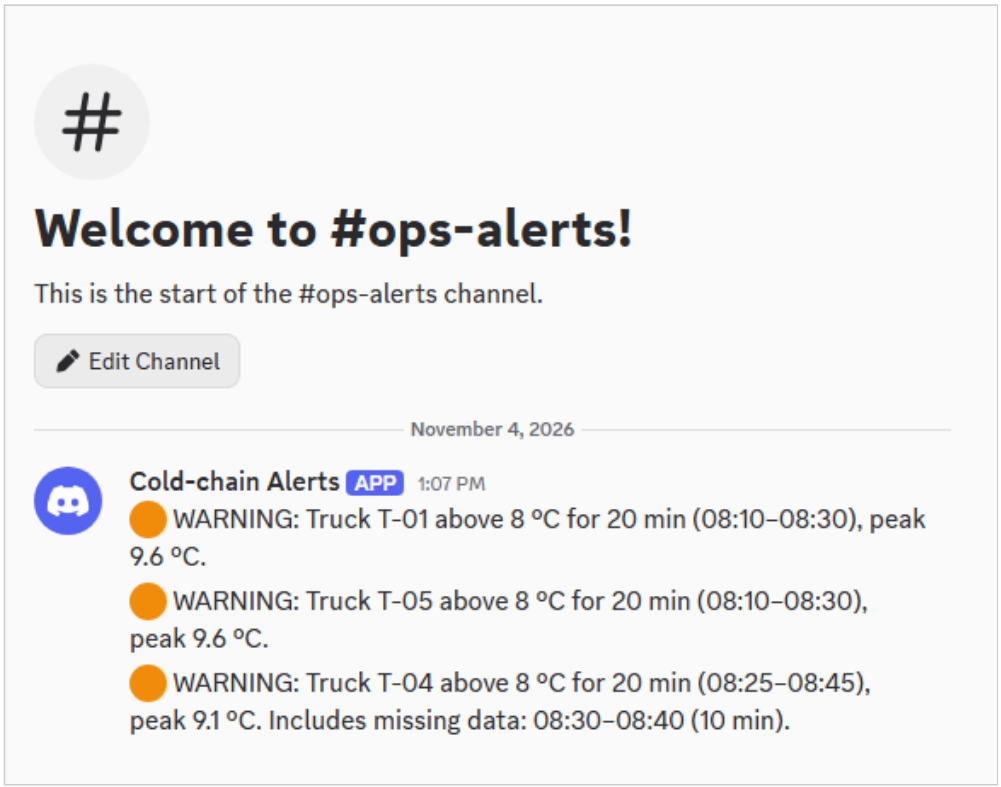


Figure 2. The three seed-data alerts delivered once each to a Discord #ops-alerts channel by the webhook notifier.

Appendix A. Assumptions (full list)

Working list of every assumption behind the alerting rules. Each one says what I assumed, why, and how it plays out on the seed data (`starter_kit/data/sensor_readings.json`, 54 readings, T-01 to T-05, 08:00 to 08:50 on 2026-07-20). The short version goes into SOLUTION_DESIGN.md.

1. Data

#	Assumption	Why
D1	Each reading has `truck_id`, `timestamp`, `temperature_c`, `humidity_pct`. A reading missing any of these, or with a value that isn't a number, is rejected and logged rather than guessed at.	Silently filling in a value could create or hide an alert.
D2	`timestamp` is the time the **sensor** took the reading, not the time we received it. All durations come from this time.	Trucks lose signal and upload in bursts. Arrival time says nothing about how long the cargo was warm.
D3	Timestamps without a time zone are treated as UTC.	The seed data has no zone. One fixed rule beats guessing per truck.
D4	The file order means nothing. Readings are sorted per truck by `timestamp` before anything else happens.	T-05 is shuffled in the file. See section 4.
D5	Two readings with the same `truck_id` and `timestamp` are duplicates: the first is kept and the second is dropped and counted.	A resent reading should not count twice. (There are none in the seed data, but retries make them likely.)
D6	A temperature below −40 °C or above 50 °C is a sensor fault, not real cargo temperature. It is flagged and left out of the alert logic.	Reefer cargo can't get to those temperatures. A faulty sensor needs a different response (fix the sensor) than a warm truck (save the cargo).
D7	Humidity is stored and shown but does not trigger alerts. A bad humidity value is dropped (set to empty) without rejecting the temperature reading.	Priya asked about temperature, and nobody gave a humidity limit. In the seed data humidity moves with temperature: correlation **0.89** across the fleet (T-01 0.98, T-02 0.88, T-03 0.94, T-04 0.91, T-05 0.98), range 48–62 %. So a humidity alert would just duplicate the temperature alert. **Caveat:** that's 54 synthetic readings over one hour. Real humidity can rise on its own (wet cargo, defrost condensation, leaks). We keep it to explain alerts, and moisture-sensitive cargo can get an optional humidity limit in its own range settings later.

2. Thresholds

#	Assumption	Why
T1	The default safe range is **2–8 °C** (each truck can get its own, see T3). A reading is out of range if it is **above 8.0** or **below 2.0**. Exactly 8.0 is in range.	2–8 °C is the standard range for refrigerated medicines, the strictest of Nimbus's cargo. Every reading in the seed data fits this profile. There's no cargo-type field, so one fleet-wide range is the honest default.

#	Assumption	Why
T2	Below 2 °C counts as out of range too.	Freezing ruins vaccines and fresh food just as heat does. No seed reading goes below 2 °C, so this has no effect on the results here.
T3	Defaults live in one config file (`backend/engine/config.py`). Every truck is **registered in the database the first time it reports, with an empty range**. Empty means the default 2–8 °C applies. Ops can fill in that truck's own range later from the dashboard (admin key needed), or send it with the truck's first upload; an upload never overwrites a range ops already set.	Different cargo (frozen at −18 °C, produce at 0–5 °C) needs different numbers, and ops shouldn't need a developer to change them. Defaulting to the strictest range means a new or forgotten truck is over-protected rather than unprotected.

3. "Too long", gaps and repeat alerts

#	Assumption	Why
A1	An **episode** starts with the first out-of-range reading and lasts until the truck has been **back in range for 10 minutes** (A7).	One problem, one episode. A single dip under 8 °C doesn't end it.
A2	**Episode duration = latest out-of-range timestamp − first out-of-range timestamp.** Not number of readings × 5 min.	The starter code counts readings, so a missing reading makes an episode look shorter (T-04 shows up as 15 min instead of 20).
A3	Alert once an episode reaches **≥ 20 minutes**.	On the seed data any limit from 10 to 20 min is correct: 5 min alerts on T-04's early blip, and 25 min misses T-04. 20 is the most patient correct value. It ignores door openings and defrost cycles (5–15 min), still has 3 real readings if one is missing, and leaves ops time to act before pharma cargo spoils.
A4	A gap of **up to 15 minutes** between two out-of-range readings does not break the episode. We assume the truck stayed warm during the gap. We never make up a temperature.	Missing one or two readings is normal. For pharma cargo, "assume it stayed warm" is the safe choice.
A5	If a truck sends nothing for **more than 15 minutes**, it gets a separate **"sensor silent"** status. If it's out of range on **both** sides of the gap, the episode covers the gap and the alert says "includes N min without data". If one side is in range, the episode stops at the last out-of-range reading.	Silence is a problem of its own (dead sensor, no signal). Warm before and after most likely means warm throughout, so for medicine we assume the worse case and say so.
A6	**One alert per episode.** While it is open, new readings update the duration and peak temperature. Only the "not improving" update (A11) or Critical (A8) sends anything new.	This is Priya's "not the same issue re-alerting every few minutes".
A7	The truck is back to normal only after **10 minutes in range since the last out-of-range reading, confirmed by at least 2 in-range readings**. This applies in SUSPECT (resets the timer) and in RECOVERING (closes the alert). Out of range again before that = the **same** episode.	Without it, a truck reading 8.5 / 7.9 / 8.5 / 7.9 would never alert, and an open alert would flap open/closed.
A8	Severity is **Warning** when the alert opens and becomes **Critical** if the temperature goes 3 °C or more out of range (≥ 11 °C) or the truck has been **out of range for 60 minutes** or more. Critical replaces any further "not improving" updates (A11). A very hot episode still waits the 20 minutes (one hot reading could be a glitch), then opens directly as Critical with a single notification. A silent-sensor notification is Critical if the truck was out of range when it went quiet, otherwise Warning.	An alert still open after an hour means the first one didn't get the problem fixed, so it goes further up. That hot usually means a failed reefer unit, so we don't wait. It's 3× the first limit, so clearly a separate step.

#	Assumption	Why
A9	Notifications go to a **webhook URL** (`ALERT_WEBHOOK_URL`, Discord or Slack), only on open and on Critical. Each alert has a fixed id (truck + start time) and sent ids are saved, so nothing is sent twice. Timeout 5 s, 3 retries; on failure we log it and the alert stays on the web page. With no URL set (and in tests), alerts go to the console.	One HTTP POST is the simplest real notification. The URL is a secret, kept only in `.env` / Secret Manager.
A11	**"Not improving" update, decided by the data with no acknowledge button.** Every 10 min after the alert opens, while it is still out of range and not yet Critical, compare the current temperature with the reading at the previous check (10 min earlier). If it hasn't moved back toward the safe range by at least **0.3 °C**, send one update: "T-04 still above 8 °C after 30 min and not improving (9.1 → 9.4 °C)". Improving, back in range (RECOVERING) or Critical: send nothing. Each update has a fixed id (alert id + check number). If a check falls in a data gap, it happens at the next out-of-range reading, and the following check is a full 10 min after that one. Seed data: T-01/T-05 drop from 9.3 to 8.2 °C between 08:30 and 08:40, and T-04 is back in range at 08:50, so there are no updates.	Without it, nothing is sent for 40 min between the alert (20 min) and Critical (60 min), even if nobody is fixing the truck. A timed repeat would bring back the noise, so the data decides: falling temperature means someone is on it, or it's recovering by itself. 0.3 °C is above normal sensor jitter (0.1–0.2 °C). At most 3 updates before Critical, and only for a truck that isn't getting better. We chose this over an acknowledge button: it needs no database or UI, works on the seed data, and stays inside the testable engine.
A10	**Silent sensor.** No reading for 15 min: page shows "sensor silent", no notification. No reading for **30 min**: one webhook ("T-03 has sent no data for 30 min, last reading 6.8 °C at 08:35"). If the truck was **SUSPECT or ALERT** when it went quiet, notify at **15 min**. One notification per silence, with a fixed id (truck + last reading time). It closes when data returns. "Now" is the clock when live, and the latest timestamp in the file for a batch run (08:50 in the seed data, so no truck is silent there; a test covers it).	Short signal drops (tunnels, rural roads) are normal, so 15 min would be noisy. 30 min is still early enough for ops to call the driver. A warm truck that has gone dark is the riskiest case, so it gets the earlier warning.

4. Out-of-order data (what to do when data isn't sorted)

The problem. T-05 has the same values as T-01, but the file lists them in this order: 08:00, 08:05, 08:20, 08:45, 08:35, 08:10, 08:50, 08:30, 08:15, 08:40, 08:25. The starter code walks the list as it comes. So it sees warm, cool, warm, warm, cool… and breaks one 30-minute problem into three short pieces (0, 5 and 15 min). **None reaches 20 minutes, so the truck that is actually as warm as T-01 gets no alert.**

What we do (v1, whole file at once):

1. Group readings by truck.
2. Sort each truck's readings by sensor `timestamp`.
3. Drop duplicates (D5) and flag bad values (D6).
4. Only then run the episode logic.

After sorting, T-05 looks exactly like T-01: out of range from 08:10 to 08:40, 30 minutes, one alert.

What we'd do live, when readings arrive one at a time and some come late:

- Keep each truck's recent readings in timestamp order. A late reading goes into its correct place, not at the end.
- When a reading arrives, recompute that truck's episode from its readings in time order. That way the result is the same whatever order readings arrive in.
- Accept late readings up to **60 minutes** old. Anything older is logged as "too late" and doesn't change any alert, so the alert history stays stable.
- If a late reading makes an episode long enough, the alert fires then, and the alert says the delay was caused by late data.
- If a late reading shows an alert that already went out should not have fired, we don't delete it. We add a note saying it was revised, so ops can see what changed and why.

5. Worked example: what happens at 25 minutes

"25 minutes" can mean two different questions. Both are answered here.

(a) A truck has been out of range for 25 minutes (limit still 20).

Example: out of range from 08:10 on, one reading every 5 minutes.

Time	Out of range for	What happens
08:10	0 min	Episode starts. No alert yet.
08:20	10 min	Still waiting.
08:30	**20 min**	**Alert opens (Warning).** Ops gets one notification.
08:35	25 min	**No new alert.** The open alert now shows 25 min and the current peak.
08:40+	30+ min	Same: the alert updates quietly. It notifies again only if it becomes Critical (≥ 11 °C, or open 60 min).
back in range for 10 min	—	Alert closes. A later warm reading starts a new episode.

So 25 minutes doesn't mean another alert. It's one open alert that has been warm for 25 minutes.

(b) The panel changes the limit to 25 minutes.

Truck	Out of range for (by timestamp)	Limit 20 min	Limit 25 min
T-01	08:10 → 08:40 = 30 min	Alert at 08:30	Alert at 08:35
T-02	single reading = 0 min	No	No
T-03	never out of range	No	No
T-04	08:25 → 08:45 = 20 min (08:35 missing)	**Alert at 08:45**	**No alert**
T-05	same as T-01 once sorted = 30 min	Alert at 08:30	Alert at 08:35

T-04 is the edge case. It meets the 20-minute limit exactly. With a 25-minute limit it drops out, because it is back in range (7.2 °C) at 08:50. This trade-off is what the panel will ask about:

- A longer limit means fewer false alarms, but slower alerts and T-04 is missed.
- A shorter limit catches T-04 sooner, but more blips risk becoming alerts.
- For pharma cargo, missing a real problem costs more than one extra alert, so I'd keep 20 minutes. It's one config value, so Nimbus can change it per cargo type without touching code.

6. What the seed data gives with these assumptions

Truck	Result	Reason
T-01	**Alert** (Warning), 08:10–08:40, 30 min, peak 9.6 °C, closes at 08:50	Sustained warming
T-02	No alert	One-reading blip (8.9 °C at 08:10)
T-03	No alert	Peaks at 7.0 °C, never out of range
T-04	**Alert** (Warning) at 08:45, 08:25–08:45, 20 min, peak 9.1 °C; still RECOVERING when the data ends	Sustained, across a missing 08:35 reading. The earlier 08:05–08:10 blip stays SUSPECT and resets at 08:20 (10 min in range).
T-05	**Alert** (Warning), same as T-01	Only visible after sorting

The starter code finds only T-01.

7. Open questions for Nimbus (we'd ask Priya)

- How long can a truck normally go without signal on its routes? (This sets the 30-min silent limit.)
- What cargo is on each truck, and what are its real limits?
- How fast does ops need to know? Is 20 minutes OK for pharma?
- How should ops be notified (dashboard, SMS, email), and who acknowledges an alert?
- How late can sensor data realistically arrive after a signal drop?

Appendix B. Testing and verification

221 automated tests: 212 Python, 7 frontend and 2 end-to-end browser tests. On top of those, static checks, dependency audits and live checks against the real Discord and Groq. **No automated test touches the network:** `tests/conftest.py` forces the webhook off and the LLM to mock mode, even if `.env` holds real keys.

How to run

```
.\.venv\Scripts\python -m pytest                                        # unit + integration, with coverage
cd frontend; npm run typecheck; npm test; npm run build; cd ..       # types, UI tests, build
.\.venv\Scripts\python -m pytest tests_e2e --browser-channel chrome --no-cov   # end-to-end (needs the build)
.\.venv\Scripts\ruff check .                                        # lint + security rules (bandit "S")
.\.venv\Scripts\pip-audit -r requirements.txt; cd frontend; npm audit --omit=dev
```

The test pyramid

Level	What it means here	Files	Tests
Unit	One piece on its own, with fakes for anything outside it (no real network, files or browser)	`test_loader`, `test_state_machine`, `test_notify`, `test_ai`	133
Integration	Several real pieces together: the real seed file, the real HTTP layer, the real database, the real MCP server	`test_seed_data`, `test_cli`, `test_api`, `test_mcp_server`, frontend `App.test.tsx`	45 + 3
End-to-end	The whole system as a user sees it: real server, real database, built dashboard, real Chrome	`tests_e2e/test_dashboard.py`	2
Live checks (manual, recorded)	Against the real outside services	Discord webhook, Groq LLM, screenshot	—

Unit tests (133)

`test_loader.py` (34): turning raw JSON into clean, sorted readings

- Sorting by sensor time; naive timestamps treated as UTC; `+05:30` converted to UTC.
- Duplicates: an exact repeat, and the same moment written in two time zones. The first one is kept.
- Impossible temperatures (−41, 50.1, 999 °C) become sensor faults. The limits themselves (−40, 50) are valid.
- 12 kinds of bad input are each rejected with a reason: not an object, missing or blank truck id, an over-long id, missing or garbage timestamp, a number given as a timestamp, a temperature given as a string, `true`, `NaN`, `Infinity`.
- Bad humidity is dropped without losing the temperature reading.

`test_state_machine.py` (45): the alerting rules, one rule per test

- **Blips and limits:** a 1-reading blip never alerts; 15 min doesn't alert; **exactly 20 min alerts**; **exactly 8.0 °C and 2.0 °C are in range**; too cold alerts; duration comes from timestamps, not reading counts.
- **Recovery:** the 8.5/7.9 sawtooth still alerts; SUSPECT resets after 10 min in range; one in-range reading doesn't close an alert; the alert closes after 10 min and 2 readings; flapping stays the same alert; a new episode gets a new alert.
- **Gaps:** one missing reading is bridged and reported (T-04); a 15-min gap still bridges; a long gap with both sides warm is bridged and flagged; a long gap with one side in range resets SUSPECT; an open alert never closes because of missing data.
- **Silent sensor:** 20 min quiet shows on the page but doesn't notify; 30 min notifies once; a warm truck going quiet notifies at 15 min as Critical.

- **Severity and updates:** opens directly as Critical at ≥ 11 °C; escalates at 60 min exactly once; a stuck truck gets "not improving" updates at 30/40/50 min, then Critical at 60; a 0.2 °C wobble counts as not improving, while **exactly 0.3 °C counts as improving**; no update while recovering; a check after a gap isn't followed by a second one 5 min later.
- **Config:** 7 bad settings are refused; the limit is configurable (25 min drops T-04).

`test_notify.py` (26): delivery to Discord/Slack, using a fake HTTP transport

- The payload works for Discord (`content`) and Slack (`text`), with a red icon for Critical, the AI text with the facts underneath, and text cut to Discord's limit.
- Retries on 500/503/429/connection error/timeout. **No** retry on 400/401/404. Gives up after 3 attempts.
- Send-once: a second run skips ids already sent; a failed send is retried next run and doesn't block the others; a broken AI never blocks an alert.
- **Security:** HTTPS only; the URL never appears in errors, `repr` or logs. That last test **found a real leak**: the HTTP library was logging the full webhook URL.

`test_ai.py` (28): AI wording and its safety check

- The facts sent to the LLM are numbers only, as of the notification time (no later readings).
- The check rejects an invented temperature (9.8), an invented time (08:20), an invented duration (25), the wrong truck, text over 300 characters, and empty text.
- Caching (one LLM call per notification), the timeout or error fallback to the plain message, the Groq/Gemini request shapes, API keys only in headers (never in URLs or error text), and the primary-to-fallback switch inside the 5 s budget.

Integration tests (45 Python + 3 frontend)

- **`test_seed_data.py` (11):** the real seed file through the loader and engine. **One test per truck story** (T-01 to T-05). Shuffling the whole file 5 ways gives an identical result. A 25-minute limit drops T-04. The starter code finds only T-01, which documents why it was replaced.
- **`test_cli.py` (8):** the command-line tool as a user runs it: the text report, `--json`, overrides, `--now` (silent sensors), listing of data problems, and clean errors (exit code 2) for a missing file, a bad time or a bad config.
- **`test_api.py` (16):** real HTTP requests through FastAPI to the engine and a **real SQLite database**.
- All the endpoints.
- **Upload → alert → exactly one push.** Re-uploading sends nothing.
- **Sent ids survive a server restart.**
- Per-reading rejection; uploads that move time forward raise silent-sensor notices.
- 422 for malformed bodies; **413 for over 5,000 readings or over 2 MB.**
- **`test_mcp_server.py` (10):** the MCP tools through the real MCP server object.
- "Which trucks are in trouble" gives T-04 recovering.
- Exactly 5 read-only tools are exposed.
- The history window is capped (0, −5 and 1441 minutes are refused); an unknown truck is refused, including an SQL-injection-style string.
- **`frontend/src/App.test.tsx` (3):** the dashboard component with the API mocked. It checks the banner, state badges, reasons, the **labelled AI summary**, the chart on click, and the error message when the API is down.

Truck ranges (`test_truck_ranges.py`, 17: unit + integration)

- **A new truck is registered with an empty range and uses the default**, only the first time; it is registered with the time it was added. An unknown truck also gets the default.
- Fill a range in later and clear it; set the alert time alone; ranges survive a restart.
- Refused: min above max, only one limit given, a range outside the sensor limits, a zero alert time, an over-long truck id.
- The same −10 °C reading is "too warm" for a frozen truck and "too cold" for a pharma truck.
- Through the API: the seed trucks are registered with empty ranges; giving T-03 a 0–5 °C range immediately creates an "Above 5 °C" alert, and clearing it removes the alert; a new truck in an upload is registered and

can bring its own range; **an upload never overwrites a range ops already set**; writes without the key get 401; bad input gets 422.
- UI tests: the panel lists every registered truck, shows an empty range as "not set (default 2 to 8 °C)", shows the time each was added, sends the admin key in the header, and shows "Wrong or missing admin key" on a 401.

Ask box over MCP (`test_ask.py`, 17)

- The answer check: numbers and times must appear in the tool results (also inside ISO timestamps); small counts allowed; an invented 9.8 °C, 07:55 or "45 minutes" is rejected.
- The agent loop with a fake MCP session: the LLM calls a tool and answers; an invented number is withheld; an unknown tool name is never called; at most 4 steps; an empty answer is handled.
- **A real MCP round trip:** the backend starts our MCP server as a separate process and talks the MCP protocol to it (offline mock LLM): the answer is "T-04 (recovering, peak 9.1 °C)" via `get_open_alerts`. An LLM outage gives a friendly message.
- The MCP server reads the live API when told to; Groq's tool-calling request shape; the endpoint (live-data URL passed to MCP), question length limits (422), the rate limit (429), and no admin key needed.

End-to-end tests (2)

`tests_e2e/test_dashboard.py` starts the **real server** (real database, built dashboard) and drives **Chrome**:

1. The page shows the open-alert banner, 3 alerts with AI summaries, and T-04's chart with the alert period shaded.
2. **The full journey:** upload warm readings for a new truck through the API → the engine opens an alert → **exactly one** push goes to the (fake) Discord sender, AI-worded → the dashboard shows T-09 in **Alert** → re-uploading sends nothing.

Live checks (manual, against the real services)

Check	Result
`python -m backend.cli --send` → real Discord webhook	3/3 delivered to `#ops-alerts` (screenshot)
`python -m backend.cli --ai` → real Groq	3 summaries, all passed the invented-number check
Server + PostgreSQL 16 + Chrome	Dashboard renders with real Groq summaries (screenshot)

Static checks and security

Check	Result
`ruff check .` (style, bugs, and the bandit `S` security rules)	All checks passed
`npm run typecheck` (TypeScript strict)	No errors; the page and API types agree
`pip-audit -r requirements.txt`	No known vulnerabilities
`npm audit --omit=dev`	0 vulnerabilities
Secret scan of every commit (API keys, webhook URL, DB password)	None committed; `.env` is git-ignored

Bugs the tests caught (and fixed)

1. **Floating-point:** a drop from 9.6 to 9.3 °C computed as 0.2999 °C, so a real improvement looked like "not improving" → the value is now rounded before comparing.
2. **Gap timing:** after a data gap, a late "is it improving?" check was followed by another only 5 min later → the next check is now a full 10 min after the actual one.
3. **Secret leak:** httpx logged the full webhook URL, which contains the token → its logger is held at WARNING.

4. **Test isolation:** importing the API module built the app with the developer's real `.env` (live Groq calls) → app creation moved to `backend/main.py`.
5. **Database password in `repr`:** the settings object printed the full `DATABASE_URL`, password included → it is now a secret value too.

Not covered yet

- Load and concurrency testing (one process; the `Fleet` lock is tested only by design, not under stress).
- Real PostgreSQL in CI: the API tests use SQLite through the same SQLAlchemy code. PostgreSQL was checked live, but not in an automated test.
- Mobile layout and accessibility audits of the dashboard.